Quick-Sort

Although its O(n2)-time worst-case performance makes quick-sort susceptible in real-time applications where we must make guarantees on the time needed to complete a sorting operation, we expect its performance to be O(nlogn)-time, and experimental studies have shown that it outperforms both heap-sort and merge-sort on many tests. Quick-sort does not naturally provide a stable sort, due to the swapping of elements during the partitioning step.

For decades quick-sort was the default choice for a general-purpose, in-memory sorting algorithm. Quick-sort was included as the qsort sorting utility provided in C language libraries, and was the basis for sorting utilities on Unix operating systems for many years. It was also the standard algorithm for sorting arrays in Java through version 6 of that language. (We discuss Java7 below.)

Merge-Sort

Merge-sort runs in O(nlogn) time in the worst case. It is quite dif·cult to make merge-sort run in-place for arrays, and without that optimization the extra overhead of allocate a temporary array, and copying between the arrays is less attractive than in-place implementations of heap-sort and quick-sort for sequences that can ·t entirely in a computer·s main memory. Even so, merge-sort is an excellent algorithm for situations where the input is strati·ed across various levels of the computer·s memory hierarchy (e.g., cache, main memory, external memory). In these contexts, the way that merge-sort processes runs of data in long merge streams makes the best use of all the data brought as a block into a level of memory, thereby reducing the total number of memory transfers.

The GNU sorting utility (and most current versions of the Linux operating system) relies on a multiway merge-sort variant. Since 2003, the standard sort method of Python·s list class has been a hybrid approach named Tim-sort (designed by Tim Peters), which is essentially a bottom-up merge-sort that takes advantage of some initial runs in the data while using insertion-sort to build additional runs. Tim-sort has also become the default algorithm for sorting arrays in Java7.

Bucket-Sort and Radix-Sort

Finally, if an application involves sorting entries with small integer keys, character strings, or d-tuples of keys from a discrete range, then bucket-sort or radix-sort is an excellent choice, for it runs in O(d(n+N)) time, where [0,N ·1] is the range of integer keys (and d = 1 for bucket sort). Thus, if d(n+N) is signi·cantly ·below· the nlogn function, then this sorting method should run faster than even quick-sort, heap-sort, or merge-sort.

12.1
Why Study Sorting Algorithms?
Much of this chapter focuses on algorithms for sorting a collection of objects. Given a collection, the goal is to rearrange the elements so that they are ordered from smallest to largest (or to produce a new copy of the sequence with such an order). As we did when studying priority queues (see Section 9.4), we assume that such a consistent order exists. In Python, the natural order of objects is typically1 de·ned using the < operator having following properties:

· Irre·exive property: k ·< k.
· Transitive property: if k1 < k2 and k2 < k3, then k1 < k3.

The transitive property is important as it allows us to infer the outcome of certain comparisons without taking the time to perform those comparisons, thereby leading to more ef·cient algorithms.

Sorting is among the most important, and well studied, of computing problems. Data sets are often stored in sorted order, for example, to allow for ef·cient searches with the binary search algorithm (see Section 4.1.3). Many advanced algorithms for a variety of problems rely on sorting as a subroutine.

Python has built-in support for sorting data, in the form of the sort method of the list class that rearranges the contents of a list, and the built-in sorted function that produces a new list containing the elements of an arbitrary collection in sorted order. Those built-in functions use advanced algorithms (some of which we will describe in this chapter), and they are highly optimized. A programmer should typically rely on calls to the built-in sorting functions, as it is rare to have a special enough circumstance to warrant implementing a sorting algorithm from scratch.

With that said, it remains important to have a deep understanding of sorting algorithms. Most immediately, when calling the built-in function, it is good to know what to expect in terms of ef·ciency and how that may depend upon the initial order of elements or the type of objects that are being sorted. More generally, the ideas and approaches that have led to advances in the development of sorting algorithm carry over to algorithm development in many other areas of computing.

We have introduced several sorting algorithms already in this book:

· Insertion-sort (see Sections 5.5.2, 7.5, and 9.4.1)
· Selection-sort (see Section 9.4.1)
· Bubble-sort (see Exercise C-7.38)
· Heap-sort (see Section 9.4.2)

In this chapter, we present four other sorting algorithms, called merge-sort, quick-sort, bucket-sort, and radix-sort, and then discuss the advantages and disadvantages of the various algorithms in Section 12.5.

1In Section 12.6.1, we will explore another technique used in Python for sorting data according to an order other than the natural order de·ned by the < operator.

12.5
Comparing Sorting Algorithms
At this point, it might be useful for us to take a moment and consider all the algorithms we have studied in this book to sort an n-element sequence.
Considering Running Time and Other Factors
We have studied several methods, such as insertion-sort, and selection-sort, that have O(n2)-time behavior in the average and worst case. We have also studied several methods with O(nlogn)-time behavior, including heap-sort, merge-sort, and quick-sort. Finally, the bucket-sort and radix-sort methods run in linear time for certain types of keys. Certainly, the selection-sort algorithm is a poor choice in any application, since it runs in O(n2) time even in the best case. But, of the remaining sorting algorithms, which is the best?
As with many things in life, there is no clear ·best· sorting algorithm from the remaining candidates. There are trade-offs involving ef·ciency, memory usage, and stability. The sorting algorithm best suited for a particular application depends on the properties of that application. In fact, the default sorting algorithm used by computing languages and systems has evolved greatly over time. We can offer some guidance and observations, therefore, based on the known properties of the ·good· sorting algorithms.
Insertion-Sort
If implemented well, the running time of insertion-sort is O(n + m), where m is the number of inversions (that is, the number of pairs of elements out of order). Thus, insertion-sort is an excellent algorithm for sorting small sequences (say, less than 50 elements), because insertion-sort is simple to program, and small sequences necessarily have few inversions. Also, insertion-sort is quite effective for sorting sequences that are already ·almost· sorted. By ·almost,· we mean that the number of inversions is small. But the O(n2)-time performance of insertion-sort makes it a poor choice outside of these special contexts.
Heap-Sort
Heap-sort, on the other hand, runs in O(nlogn) time in the worst case, which is optimal for comparison-based sorting methods. Heap-sort can easily be made to execute in-place, and is a natural choice on small- and medium-sized sequences, when input data can ·t into main memory. However, heap-sort tends to be outperformed by both quick-sort and merge-sort on larger sequences. A standard heap-sort does not provide a stable sort, because of the swapping of elements.

12.4 Studying Sorting through an Algorithmic Lens

Recapping our discussions on sorting to this point, we have described several methods with either a worst case or expected running time of O(nlogn) on an input sequence of size n. These methods include merge-sort and quick-sort, described in this chapter, as well as heap-sort (Section 9.4.2). In this section, we study sorting as an algorithmic problem, addressing general issues about sorting algorithms.

12.4.1 Lower Bound for Sorting

A natural ·rst question to ask is whether we can sort any faster than O(nlogn) time. Interestingly, if the computational primitive used by a sorting algorithm is the comparison of two elements, this is in fact the best we can do·comparison-based sorting has an ·(nlogn) worst-case lower bound on its running time. (Recall the notation ·(·) from Section 3.3.1.) To focus on the main cost of comparison-based sorting, let us only count comparisons, for the sake of a lower bound.

Suppose we are given a sequence S = (x0,x1,...,xn·1) that we wish to sort, and assume that all the elements of S are distinct (this is not really a restriction since we are deriving a lower bound). We do not care if S is implemented as an array or a linked list, for the sake of our lower bound, since we are only counting comparisons. Each time a sorting algorithm compares two elements xi and xj (that is, it asks, ·is xi < xj?·), there are two outcomes: ·yes· or ·no.· Based on the result of this comparison, the sorting algorithm may perform some internal calculations (which we are not counting here) and will eventually perform another comparison between two other elements of S, which again will have two outcomes. Therefore, we can represent a comparison-based sorting algorithm with a decision tree T (recall Example 8.6). That is, each internal node v in T corresponds to a comparison and the edges from position v to its children correspond to the computations resulting from either a ·yes· or ·no· answer. It is important to note that the hypothetical sorting algorithm in question probably has no explicit knowledge of the tree T. The tree simply represents all the possible sequences of comparisons that a sorting algorithm might make, starting from the ·rst comparison (associated with the root) and ending with the last comparison (associated with the parent of an external node). Each possible initial order, or permutation, of the elements in S will cause our hypothetical sorting algorithm to execute a series of comparisons, traversing a path in T from the root to some external node. Let us associate with each external node v in T, then, the set of permutations of S that cause our sorting algorithm to end up in v. The most important observation in our lower-bound argument is that each external node v in T can represent the sequence of comparisons for at most one permutation of S. The justi·cation for this claim is simple: If two different

Projects

P-12.56 Implement a nonrecursive, in-place version of the quick-sort algorithm, as described at the end of Section 12.3.2.

P-12.57 Experimentally compare the performance of in-place quick-sort and a version of quick-sort that is not in-place.

P-12.58 Perform a series of benchmarking tests on a version of merge-sort and quick-sort to determine which one is faster. Your tests should include sequences that are ·random· as well as ·almost· sorted.

P-12.59 Implement deterministic and randomized versions of the quick-sort algorithm and perform a series of benchmarking tests to see which one is faster. Your tests should include sequences that are very ·random· looking as well as ones that are ·almost· sorted.

P-12.60 Implement an in-place version of insertion-sort and an in-place version of quick-sort. Perform benchmarking tests to determine the range of values of n where quick-sort is on average better than insertion-sort.

P-12.61 Design and implement a version of the bucket-sort algorithm for sorting a list of n entries with integer keys taken from the range [0,N ·1], for N ·2. The algorithm should run in O(n+N) time.

P-12.62 Design and implement an animation for one of the sorting algorithms described in this chapter. Your animation should illustrate the key properties of this algorithm in an intuitive manner.

Chapter Notes

Knuth·s classic text on Sorting and Searching [65] contains an extensive history of the sorting problem and algorithms for solving it. Huang and Langston [53] show how to merge two sorted lists in-place in linear time. The standard quick-sort algorithm is due to Hoare [51]. Several optimizations for quick-sort are described by Bentley and McIlroy [16]. More information about randomization, including Chernoff bounds, can be found in the appendix and the book by Motwani and Raghavan [80]. The quick-sort analysis given in this chapter is a combination of the analysis given in an earlier Java edition of this book and the analysis of Kleinberg and Tardos [60]. Exercise C-12.32 is due to Littman. Gonnet and Baeza-Yates [44] analyze and compare experimentally several sorting algorithms. The term ·prune-and-search· comes originally from the computational geometry literature (such as in the work of Clarkson [26] and Megiddo [75]). The term ·decrease-and-conquer· is from Levitin [70].

5.5.2
Sorting a Sequence
In the previous subsection, we considered an application for which we added an object to a sequence at a given position while shifting other elements so as to keep the previous order intact. In this section, we use a similar technique to solve the sorting problem, that is, starting with an unordered sequence of elements and rearranging them into nondecreasing order.
The Insertion-Sort Algorithm
We study several sorting algorithms in this book, most of which are described in Chapter 12. As a warm-up, in this section we describe a nice, simple sorting algorithm known as insertion-sort. The algorithm proceeds as follows for an array-based sequence. We start with the ·rst element in the array. One element by itself is already sorted. Then we consider the next element in the array. If it is smaller than the ·rst, we swap them. Next we consider the third element in the array. We swap it leftward until it is in its proper order with the ·rst two elements. We then consider the fourth element, and swap it leftward until it is in the proper order with the ·rst three. We continue in this manner with the ·fth element, the sixth, and so on, until the whole array is sorted. We can express the insertion-sort algorithm in pseudo-code, as shown in Code Fragment 5.9.
Algorithm InsertionSort(A):
Input: An array A of n comparable elements
Output: The array A with elements rearranged in nondecreasing order
for k from 1 to n ·1 do
Insert A[k] at its proper location within A[0], A[1], ..., A[k].
Code Fragment 5.9: High-level description of the insertion-sort algorithm.
This is a simple, high-level description of insertion-sort. If we look back to Code Fragment 5.8 of Section 5.5.1, we see that the task of inserting a new entry into the list of high scores is almost identical to the task of inserting a newly considered element in insertion-sort (except that game scores were ordered from high to low). We provide a Python implementation of insertion-sort in Code Fragment 5.10, using an outer loop to consider each element in turn, and an inner loop that moves a newly considered element to its proper location relative to the (sorted) subarray of elements that are to its left. We illustrate an example run of the insertion-sort algorithm in Figure 5.20.
The nested loops of insertion-sort lead to an O(n2) running time in the worst case. The most work is done if the array is initially in reverse order. On the other hand, if the initial array is nearly sorted or perfectly sorted, insertion-sort runs in O(n) time because there are few or no iterations of the inner loop.

12.4.2
Linear-Time Sorting: Bucket-Sort and Radix-Sort
In the previous section, we showed that ·(nlogn) time is necessary, in the worst case, to sort an n-element sequence with a comparison-based sorting algorithm. A natural question to ask, then, is whether there are other kinds of sorting algorithms that can be designed to run asymptotically faster than O(nlogn) time. Interestingly, such algorithms exist, but they require special assumptions about the input sequence to be sorted. Even so, such scenarios often arise in practice, such as when sorting integers from a known range or sorting character strings, so discussing them is worthwhile. In this section, we consider the problem of sorting a sequence of entries, each a key-value pair, where the keys have a restricted type.
Bucket-Sort
Consider a sequence S of n entries whose keys are integers in the range [0,N ·1], for some integer N ·2, and suppose that S should be sorted according to the keys of the entries. In this case, it is possible to sort S in O(n+N) time. It might seem surprising, but this implies, for example, that if N is O(n), then we can sort S in O(n) time. Of course, the crucial point is that, because of the restrictive assumption about the format of the elements, we can avoid using comparisons.
The main idea is to use an algorithm called bucket-sort, which is not based on comparisons, but on using keys as indices into a bucket array B that has cells indexed from 0 to N ·1. An entry with key k is placed in the ·bucket· B[k], which itself is a sequence (of entries with key k). After inserting each entry of the input sequence S into its bucket, we can put the entries back into S in sorted order by enumerating the contents of the buckets B[0],B[1],...,B[N ·1] in order. We describe the bucket-sort algorithm in Code Fragment 12.7.
Algorithm bucketSort(S):
Input: Sequence S of entries with integer keys in the range [0,N·1]
Output: Sequence S sorted in nondecreasing order of the keys
let B be an array of N sequences, each of which is initially empty
for each entry e in S do
k = the key of e
remove e from S and insert it at the end of bucket (sequence) B[k]
for i = 0 to N·1 do
for each entry e in sequence B[i] do
remove e from B[i] and insert it at the end of S
Code Fragment 12.7: Bucket-sort.

R-12.12 If the outermost while loop of our implementation of inplace quick sort (line 7 of Code Fragment 12.6) were changed to use condition left < right (rather than left <= right), there would be a ·aw. Explain the ·aw and give a speci·c input sequence on which such an implementation fails.
R-12.13 If the conditional at line 14 of our inplace quick sort implementation of Code Fragment 12.6 were changed to use condition left < right (rather than left <= right), there would be a ·aw. Explain the ·aw and give a speci·c input sequence on which such an implementation fails.
R-12.14 Following our analysis of randomized quick-sort in Section 12.3.1, show that the probability that a given input element x belongs to more than 2logn subproblems in size group i is at most 1/n2.
R-12.15 Of the n! possible inputs to a given comparison-based sorting algorithm, what is the absolute maximum number of inputs that could be correctly sorted with just n comparisons?
R-12.16 Jonathan has a comparison-based sorting algorithm that sorts the ·rst k elements of a sequence of size n in O(n) time. Give a big-Oh characterization of the biggest that k can be?
R-12.17 Is the bucket-sort algorithm in-place? Why or why not?
R-12.18 Describe a radix-sort method for lexicographically sorting a sequence S of triplets (k,l,m), where k, l, and m are integers in the range [0,N ·1], for some N ·2. How could this scheme be extended to sequences of d-tuples (k1,k2,...,kd), where each ki is an integer in the range [0,N ·1]?
R-12.19 Suppose S is a sequence of n values, each equal to 0 or 1. How long will it take to sort S with the merge-sort algorithm? What about quick-sort?
R-12.20 Suppose S is a sequence of n values, each equal to 0 or 1. How long will it take to sort S stably with the bucket-sort algorithm?
R-12.21 Given a sequence S of n values, each equal to 0 or 1, describe an in-place method for sorting S.
R-12.22 Give an example input list that requires merge-sort and heap-sort to take O(nlogn) time to sort, but insertion-sort runs in O(n) time. What if you reverse this list?
R-12.23 What is the best algorithm for sorting each of the following: general comparable objects, long character strings, 32-bit integers, double-precision ·oating-point numbers, and bytes? Justify your answer.
R-12.24 Show that the worst-case running time of quick-select on an n-element sequence is ·(n2).

Creativity

C-12.25 Linda claims to have an algorithm that takes an input sequence S and produces an output sequence T that is a sorting of the n elements in S.
a. Give an algorithm, is sorted, that tests in O(n) time if T is sorted.
b. Explain why the algorithm is sorted is not suf·cient to prove a particular output T to Linda·s algorithm is a sorting of S.
c. Describe what additional information Linda·s algorithm could output so that her algorithm·s correctness could be established on any given S and T in O(n) time.

C-12.26 Describe and analyze an ef·cient method for removing all duplicates from a collection A of n elements.

C-12.27 Augment the PositionalList class (see Section 7.4) to support a method named merge with the following behavior. If A and B are PositionalList instances whose elements are sorted, the syntax A.merge(B) should merge all elements of B into A so that A remains sorted and B becomes empty. Your implementation must accomplish the merge by relinking existing nodes; you are not to create any new nodes.

C-12.28 Augment the PositionalList class (see Section 7.4) to support a method named sort that sorts the elements of a list by relinking existing nodes; you are not to create any new nodes. You may use your choice of sorting algorithm.

C-12.29 Implement a bottom-up merge-sort for a collection of items by placing each item in its own queue, and then repeatedly merging pairs of queues until all items are sorted within a single queue.

C-12.30 Modify our in-place quick-sort implementation of Code Fragment 12.6 to be a randomized version of the algorithm, as discussed in Section 12.3.1.

C-12.31 Consider a version of deterministic quick-sort where we pick as our pivot the median of the d last elements in the input sequence of n elements, for a ·xed, constant odd number d ·3. What is the asymptotic worst-case running time of quick-sort in this case?

C-12.32 Another way to analyze randomized quick-sort is to use a recurrence equation. In this case, we let T(n) denote the expected running time of randomized quick-sort, and we observe that, because of the worst-case partitions for good and bad splits, we can write

T(n) ·1
2 (T(3n/4)+T(n/4)) + 1
2 (T(n·1)) +bn,

where bn is the time needed to partition a list for a given pivot and concatenate the result sublists after the recursive calls return. Show, by induction, that T(n) is O(nlogn).

```
1
def merge_sort(S):
2
  ···Sort the elements of Python list S using the merge-sort algorithm.···
3
  n = len(S)
4
  if n < 2:
5
    return
  # list is already sorted
6
  # divide
7
  mid = n // 2
8
  S1 = S[0:mid]
  # copy of ·rst half
9
  S2 = S[mid:n]
  # copy of second half
10
  # conquer (with recursion)
11
  merge_sort(S1)
  # sort copy of ·rst half
12
  merge_sort(S2)
  # sort copy of second half
13
  # merge results
14
  merge(S1, S2, S)
  # merge sorted halves back into S
```
Code Fragment 12.2: An implementation of the recursive merge-sort algorithm for Python·s array-based list class (using the merge function de·ned in Code Fragment 12.1.

12.2.3

The Running Time of Merge-Sort

We begin by analyzing the running time of the merge algorithm. Let n1 and n2 be the number of elements of S1 and S2, respectively. It is clear that the operations performed inside each pass of the while loop take O(1) time. The key observation is that during each iteration of the loop, one element is copied from either S1 or S2 into S (and that element is considered no further). Therefore, the number of iterations of the loop is n1 +n2. Thus, the running time of algorithm merge is O(n1 +n2).

Having analyzed the running time of the merge algorithm used to combine subproblems, let us analyze the running time of the entire merge-sort algorithm,